Enhanced Cross Residual Graph Neural Networks for Graph Classification

Your Name
Department/Institution
University/Organization
City, Country
email@example.com

March 29, 2026

Abstract

Deep graph classification remains challenging because repeated neighborhood aggregation can blur discriminative node patterns, while conventional single-branch architectures often struggle to preserve interactions between local substructures and global graph semantics. This paper studies Enhanced Cross Residual Graph Neural Networks (ECR-GNN), a modular framework that combines intra-branch residual reuse with cross-branch information exchange at both node and graph levels. Built on a unified message-passing pipeline, the framework enables direct comparison between plain stacking, residual accumulation, and cross-residual propagation under the same training setting. By repeatedly reusing intermediate representations and feeding graph summaries back into subsequent blocks, the proposed design alleviates over-smoothing, improves depth tolerance, and strengthens the coupling between local structural cues and whole-graph context. Experiments on benchmark graph classification tasks show that residual mechanisms provide the most consistent gains in robustness and predictive performance, while cross-branch interaction is especially effective on graphs whose labels depend on coordinated multi-region or multi-substructure patterns. The study therefore shows that cross-residual design is not a universally dominant choice, but a targeted and effective strategy for graph classification scenarios that require stable multi-scale feature preservation and stronger global-context modeling.

1 Introduction

Graph classification is a fundamental task in graph representation learning, with applications in molecular property prediction, drug discovery, social network analysis, and computer vision. Unlike node-level prediction, graph classification requires the model to summarize an entire graph into a single discriminative representation. This setting amplifies several well-known difficulties of graph neural networks (GNNs): graphs vary in size and topology, local neighborhoods can be noisy, long-range dependencies are hard to preserve, and deep message passing often suffers from over-smoothing and unstable optimization.

Modern GNNs address these challenges through a diverse set of message-passing operators. Graph Convolutional Networks (GCN) [1] emphasize smooth neighborhood aggregation, Graph

Attention Networks (GAT) [2] learn adaptive neighbor weights, and graph Transformers [3] broaden the receptive field through attention. Despite their differences, these operators are usually studied as alternative backbones inside a largely sequential architecture. As depth increases, repeated aggregation can still blur node representations and weaken the useful diversity of intermediate features [4–6].

Several research directions have tried to alleviate this problem. Residual connections [7, 8], normalization [9, 10], and edge dropout [11] stabilize deep training. Multi-scale aggregation methods such as Jumping Knowledge [12] and dense connectivity [13] preserve information from multiple depths. Pooling methods such as DiffPool [14] and SAGPool [15] add hierarchical structure. However, most of these designs still operate within a single branch and do not explicitly compare different ways of reusing information across branches and graph-level summaries within one framework.

This paper studies Enhanced Cross Residual Graph Neural Networks (ECR-GNN), a modular framework for comparing several residual and cross-residual mechanisms within graph classification. Rather than treating the method as a single monolithic model, we formulate it as a family of six related graph classifiers built from a shared message-passing backbone. The family spans plain stacking, intra-branch residual reuse, node-level cross residual, sequential graph-level conditioning, repeated graph-level residual refinement, and graph-level cross residual between block pairs. This perspective makes it possible to ask a more practical question: under what graph conditions does extra cross-branch interaction help beyond a simpler residual baseline? In particular, we are interested in whether these mechanisms are most useful when graph labels depend on coordinated local structures and whole-graph arrangement rather than on a single dominant neighborhood pattern.

The contributions of this work are summarized as follows:

- We formalize ECR-GNN as a unified family of graph classification architectures and distinguish the roles of node-level residual reuse and graph-level residual propagation within that family.
- We distinguish two reusable information-flow mechanisms: node-level cross residual between parallel branches and graph-level residual propagation between sequential blocks.
- We perform a controlled empirical study on four TUDataset benchmarks, covering six architectures, three operators, five depths, and two widths. The results show that residual reuse is the most reliable improvement, while cross-branch interaction is most effective on graph classification tasks that require stronger coupling between local substructures and global context.
- We provide a comparative analysis of accuracy, stability, and efficiency, together with a pseudocode-level description that highlights the common structure of the proposed architecture family.

2 Related Work

We review related work from five perspectives: graph neural networks for graph classification, message passing architectures and operators, deep GNN training challenges, hybrid architectures, and cross-layer aggregation mechanisms. Our work builds upon these foundations by introducing a unified framework for comparing multiple residual information-flow patterns under a shared graph classification setting.

2.1 Graph Neural Networks for Graph Classification

Graph-level prediction requires aggregating node-level features into fixed-size graph representations. Early graph neural networks [16] laid the foundation for learning on graph-structured data through recursive message passing. The Message Passing Neural Networks (MPNN) framework [17] unified various GNN architectures under a common formalism, identifying message and vertex update functions as core components.

For graph classification, readout functions play a crucial role in converting node embeddings to graph-level representations. Simple approaches apply global pooling operations such as mean or sum pooling [18]. However, these methods may not adequately capture hierarchical structural information. To address this, differentiable pooling methods have been proposed: DiffPool [14] learns node cluster assignments to coarsen graphs hierarchically, SAGPool [15] employs self-attention to identify salient nodes, and LocalPool [19] enables sparse hierarchical classification. Recent surveys provide comprehensive overviews of graph pooling techniques [20, 21]. While these approaches achieve strong performance, they primarily focus on pooling design rather than on how residual information should be reused across node-level and graph-level computation stages.

2.2 Message Passing Architectures and Operators

Various message passing operators have been developed, each imposing different inductive biases on neighborhood aggregation. Graph Convolutional Networks (GCN) [1] approximate spectral convolutions through normalized adjacency matrices, performing low-pass filtering that smooths node features. GraphSAGE [22] samples and aggregates features from local neighborhoods using mean, LSTM, or pooling aggregators, enabling inductive learning on unseen graphs. Graph Isomorphism Networks (GIN) [6] use injective aggregation functions to achieve maximum discriminative power, matching the expressive power of the Weisfeiler-Lehman test.

Attention mechanisms have been widely incorporated to weight neighbor contributions adaptively. Graph Attention Networks (GAT) [2] compute attention coefficients for each neighbor, allowing nodes to focus on relevant information. Personalized Graph Neural Networks (PGNN) [23] further extend attention mechanisms for personalized aggregation. Recent work generalizes Transformer architectures to graphs [3] and incorporates edge directionality [24, 25] for improved modeling of directed and heterophilic graphs.

Other operator variants include ARMA filters [26] for flexible frequency response, MixHop [27] for mixing multi-hop neighborhoods, and anti-symmetric convolutions [28] for stable deep architectures. While these operators are typically studied and deployed individually within

homogeneous architectures, a critical gap remains: the literature provides much less comparison of how different residual information-flow mechanisms behave across operator families under a unified training protocol.

2.3 Deep GNN Training and Representation Degradation

Training deep GNNs faces fundamental challenges due to representation degradation. The *over-smoothing* phenomenon causes node representations to converge toward indistinguishable values as depth increases [4,5,29]. Theoretical analysis [29] provides non-asymptotic bounds on over-smoothing, revealing fundamental limitations in expressive power for deep architectures. Additionally, *over-squashing* [30,31] occurs when long-range dependencies cannot be effectively propagated due to graph bottlenecks, further constraining model performance.

To enable deep training, various techniques have been proposed. Residual connections [7], originally developed for computer vision, have been adapted to GNNs through residual gated graph convolutions [8]. Normalization methods such as PairNorm [9] and GraphNorm [10] alleviate over-smoothing by normalizing node features, while learnable normalization [32] adapts to different graph distributions. Regularization techniques like DropEdge [11] randomly drop edges during training to prevent overfitting and improve generalization.

Advanced architectures attempt to train extremely deep networks: techniques for training 1000-layer GNNs [33] and simplified deep GCN approaches [34] demonstrate that depth can improve performance with proper design. However, these methods primarily focus on single-path deepening and do not explicitly compare node-level residual reuse with graph-level residual propagation as two distinct ways of preserving information at depth.

2.4 Multi-Operator and Hybrid Architectures

Combining multiple aggregation schemes or architectural components has shown promise in enhancing representational capacity. Multi-hop attention [35] enables information flow across longer distances, while ARMA filters [26] provide flexible frequency responses beyond polynomial filters. Hybrid architectures incorporate edge directionality [24, 25] to capture directed relationships, and anti-symmetric convolutions [28] improve stability in deep networks.

Surveys on attention-based GNNs [36] and graph embeddings [37] provide comprehensive taxonomies of architectural variants. DenseGNN [13] connects all layers densely, similar to DenseNet in computer vision, to maintain information flow. Hierarchical pooling methods [14, 38] create multi-scale representations through graph coarsening. However, most existing hybrid approaches emphasize component combination rather than a systematic comparison of where residual information should be injected and reused. Our work addresses this gap by contrasting node-level residual reuse, graph-level residual propagation, and cross-branch information exchange within a unified architecture family.

2.5 Cross-Layer and Multi-Scale Aggregation

Cross-layer connections aim to capture multi-scale information by aggregating features from different depths. Jumping Knowledge (JK) networks [12] adaptively select which layer's repre-

sentation to use for each node, enabling flexible neighborhood range selection. Graph U-Net [39] adopts encoder-decoder architecture with skip connections between corresponding layers.

Propagation-based methods improve information flow across multiple hops: APPNP [40] combines personalized PageRank with neural predictions, while PairNorm [9] and DropEdge [11] mitigate over-smoothing through normalization and regularization. Recent work on over-smoothing [5, 41] provides comprehensive analysis and solutions.

Despite these advances, most cross-layer methods operate within a single homogeneous branch using one type of operator. A fundamental limitation is that they rarely distinguish between residual reuse at the node-representation level and residual reuse at the graph-summary level. Our framework focuses on this distinction by introducing node-level and graph-level cross-residual mechanisms within a common graph classification design space. This perspective enables a more direct comparison of how different information-flow patterns affect robustness, depth tolerance, and graph-level discrimination.

3 Task Definition

We formally define the graph classification problem and establish notation for our cross-residual framework. Rather than binding the method to a single message-passing operator, we treat the operator choice as a modular backbone and study how different residual information-flow mechanisms affect graph classification behavior under a common formulation.

3.1 Graph Representation and Notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, where d_{in} is the input feature dimension. The node features for the entire graph are collected in a matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$, where the i-th row corresponds to $\mathbf{x}_i$.

The graph structure is represented using an adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ if there exists an edge $(v_i, v_j) \in \mathcal{E}$, and $A_{ij} = 0$ otherwise. For undirected graphs, the adjacency matrix is symmetric ($\mathbf{A} = \mathbf{A}^\top$). Optionally, edge features can be represented as $\mathbf{e}_{uv} \in \mathbb{R}^{d_e}$ for each edge $(u, v) \in \mathcal{E}$, or collected in a tensor $\mathbf{E} \in \mathbb{R}^{n \times n \times d_e}$.

3.2 Graph Classification Problem

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^{N}$ consisting of N graphs, where each graph $\mathcal{G}_i$ is associated with a label $y_i \in \mathcal{Y}$, our objective is to learn a mapping function $f_\theta : \mathcal{G} \to \hat{y}$ that predicts the label $\hat{y} \in \mathcal{Y}$ for an unseen graph. The label space $\mathcal{Y}$ depends on the task type:

- **Binary classification**: $\mathcal{Y} = \{0, 1\}$ (e.g., molecular toxicity prediction)
- **Multi-class classification**: $\mathcal{Y} = \{1, 2, \ldots, C\}$ (e.g., graph categorization into C classes)
- **Multi-label classification**: $\mathcal{Y} \subseteq \{1, 2, \ldots, C\}$ (e.g., a graph may belong to multiple categories simultaneously)

The function f_θ is parameterized by neural network weights θ and operates by first learning node-level representations through message passing, then aggregating these into a graph-level representation, and finally mapping this representation to the output label space.

3.3 Message Passing with Multiple Operators

Our framework supports multiple message passing operators, where each operator imposes a distinct inductive bias on neighborhood aggregation. Common operators include:

- **GCN** [1]: Normalized adjacency-based aggregation with low-pass filtering
- **GAT** [2]: Attention-weighted aggregation with learnable importance coefficients
- **GraphSAGE** [22]: Sampling-based aggregation with mean/LSTM/pooling operators
- **GIN** [6]: Injective aggregation for maximum discriminative power
- **Transformer** [3]: Self-attention over nodes for long-range dependencies

For a chosen operator Φ, node representations are updated over L layers. At layer ℓ, the representation $\mathbf{h}_i^{(\ell)}$ of node v_i is computed as:

$$\mathbf{h}_i^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \cdot \mathrm{AGGREGATE}_\Phi\left(\left\{\mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i)\right\}, \mathbf{h}_i^{(\ell)}\right) + \mathbf{b}^{(\ell)}\right) \tag{1}$$

where $\mathrm{AGGREGATE}_\Phi$ is the aggregation function induced by operator Φ (e.g., normalized neighborhood aggregation for GCN, attention-weighted aggregation for GAT), $\mathbf{W}^{(\ell)}$ are learnable weights, $\mathbf{b}^{(\ell)}$ is a bias term, and σ is a non-linear activation function (e.g., ReLU). The proposed framework extends this standard formulation through residual reuse mechanisms that either preserve intermediate node states within or across branches, or propagate graph-level summaries across successive computation blocks, as detailed in Section 4.

3.4 Graph-Level Representation Learning

After L layers of message passing, a readout function R aggregates node-level representations into a graph-level embedding $\mathbf{h}_\mathcal{G} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{h}_\mathcal{G} = R\left(\left\{\mathbf{h}_i^{(L)} : i = 1, \ldots, n\right\}\right) \tag{2}$$

Common readout functions include mean, max, and sum pooling. In our framework, graph-level representations may also be reused across successive computation blocks, enabling later stages to incorporate coarse whole-graph context in addition to local message-passing features.

3.5 Learning Objective

The graph-level representation is passed through a classifier (typically a linear layer followed by softmax) to produce the predicted label distribution:

$$\hat{\mathbf{y}} = \mathrm{softmax}\left(\mathbf{W}_{\text{clf}}\mathbf{h}_\mathcal{G} + \mathbf{b}_{\text{clf}}\right) \tag{3}$$

where $\hat{\mathbf{y}} \in \mathbb{R}^C$ (for multi-class classification) is the predicted probability distribution over C classes, and $\mathbf{W}_{\text{clf}} \in \mathbb{R}^{C \times d_{\text{out}}}$, $\mathbf{b}_{\text{clf}} \in \mathbb{R}^C$ are classifier parameters.

The model is trained end-to-end by minimizing the cross-entropy loss function:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{i,c} \log(\hat{y}_{i,c}) \tag{4}$$

where $y_{i,c} \in \{0, 1\}$ is the ground-truth label indicator (one-hot encoding) for graph $\mathcal{G}_i$ and class c, and $\hat{y}_{i,c}$ is the predicted probability for class c. For binary classification, this reduces to the binary cross-entropy loss. The parameters θ include all weights in the message passing layers, readout functions, cross-residual connections, and the classifier, which are optimized using gradient-based methods (e.g., Adam) with backpropagation.

4 Proposed Model

ECR-GNN is formulated as a compact family of reusable graph classification architectures. All variants share the same input projection, stacked message-passing layers, global mean pooling, and linear graph classifier; they differ only in how intermediate node states and graph-level summaries are reused across depth and across branches.

4.1 Operator Factory and Shared Backbone

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$. The operator factory instantiates one of three message-passing layers,

$$\Phi \in \{\text{GCNConv},\ \text{GATConv},\ \text{TransformerConv}\},$$

and reuses the same operator type throughout a model instance. For a hidden width d, every branch starts with an input projection

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A}))\,, \tag{5}$$

followed by L hidden layers and a readout

$$\mathbf{g} = \text{MeanPool}\Big(\mathbf{H}^{(L)}\Big)\,. \tag{6}$$

The final graph prediction is produced by

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{clf}}\, \text{Dropout}(\mathbf{g}) + \mathbf{b}_{\text{clf}}. \tag{7}$$

All six architectures differ only in how they update $\mathbf{H}^{(\ell)}$ and how they reuse graph-level embeddings across blocks.

4.2 Single-Branch and Node-Level Residual Blocks

PlainGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \mathrm{Dropout}\Big(\mathrm{ReLU}\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)}, \mathbf{A}\Big)\Big)\Big). \tag{8}$$

This model serves as the reference point for judging whether residual information flow is beneficial.

NodeResGNN. The first residual variant stores the previous hidden state within the same branch:

$$\mathbf{H}^{(\ell+1)} = \mathrm{Dropout}\Big(\mathrm{ReLU}\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}\Big)\Big)\Big). \tag{9}$$

This mechanism is lightweight and preserves historical features without creating extra branches.

NodeCrossGNN. The node-level cross-residual model maintains two parallel branches with separate input projections:

$$\mathbf{H}_1^{(\ell+1)} = \mathrm{Dropout}\Big(\mathrm{ReLU}\Big(\Phi_\ell^{(1)}\Big(\mathbf{H}_1^{(\ell)} + \widetilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\Big)\Big)\Big), \tag{10}$$

$$\mathbf{H}_2^{(\ell+1)} = \mathrm{Dropout}\Big(\mathrm{ReLU}\Big(\Phi_\ell^{(2)}\Big(\mathbf{H}_2^{(\ell)} + \widetilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\Big)\Big)\Big), \tag{11}$$

where $\widetilde{\mathbf{H}}_k^{(\ell)}$ is the cached hidden state from the opposite branch before its current update. The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \tag{12}$$

Compared with the single-branch residual block, NodeCrossGNN explicitly exchanges information between parallel pathways at every depth.

4.3 Graph-Level Residual Propagation

We further consider three higher-level architectures that compose several block models sequentially and pass graph embeddings across them.

GraphCondGNN. Two plain message-passing blocks are executed in sequence. The first block produces a graph embedding $\mathbf{g}^{(1)}$, which is added to node features and to the pooled graph representation inside the second block:

$$\mathbf{g}^{(2)} = B_2\Big(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(1)}\Big). \tag{13}$$

This introduces graph-level conditioning without intra-layer residual connections.

GraphResGNN. This variant chains three graph-conditioned blocks:

$$\mathbf{g}^{(t+1)} = B_{t+1}\Big(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}\Big), \quad t \in \{1, 2\}. \tag{14}$$

Inside each block, the graph embedding is broadcast back to node states before the next graph convolution. The design can be viewed as repeated graph-level residual refinement.

GraphCrossGNN. The strongest graph-level interaction appears in the cross-graph vari-

Input: graph batch $(\mathbf{X}, \mathbf{A}, \mathbf{b})$, operator Φ, depth L, architecture type M
Instantiate message-passing blocks with operator Φ
if $M = \textsc{PlainGNN}$ **then**
    Apply one input projection and stack L message-passing layers
else if $M = \textsc{NodeResGNN}$ **then**
    Cache the previous hidden state and add it before each layer update
else if $M = \textsc{NodeCrossGNN}$ **then**
    Build two branches and inject the opposite branch's cached state at every depth
else if $M \in \{\textsc{GraphCondGNN}, \textsc{GraphResGNN}\}$ **then**
    Execute sequential block models and reuse the previous graph embedding as graph-level context
else if $M = \textsc{GraphCrossGNN}$ **then**
    Process block pairs, compute two graph embeddings, and swap them before the next pair
end if
Pool the final node representations with global mean pooling to obtain $\mathbf{g}$
Apply dropout and a linear classifier to produce logits $\hat{\mathbf{y}}$
Return: $(\hat{\mathbf{y}}, \mathbf{g})$

Figure 1: Pseudocode of the shared ECR-GNN forward procedure

ant. Four blocks are organized as two sequential pairs. Within each stage, the two block outputs are swapped and reused as the next stage's graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = \left(B_{2t-1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}\right),\ B_{2t}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}\right)\right), \tag{15}$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = \left(\mathbf{g}_2^{(t)},\ \mathbf{g}_1^{(t)}\right). \tag{16}$$

The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \tag{17}$$

In this variant, interaction occurs through exchanged pooled graph summaries rather than through direct node-tensor exchange.

4.4 Unified Training Objective

All model variants use the same graph classification objective:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c} \log \hat{y}_{i,c}, \tag{18}$$

optimized with Adam. Dropout is applied after every message-passing update and once more before the classifier. The experiments in Section 6 therefore isolate architectural differences rather than changes in loss, optimizer, or data protocol.

4.5 Algorithmic View

Algorithm 1 summarizes the shared forward logic in pseudocode form rather than as a descriptive table.

4.6 Design Implications

This architecture family suggests three practical hypotheses that are later examined experimentally. First, residual reuse should matter more consistently than simply adding more branches, because every successful variant still relies on repeated feature reuse. Second, node-level cross residual and graph-level residual passing are not interchangeable: the former exchanges intermediate node representations, whereas the latter only re-injects pooled graph summaries. Third, the usefulness of cross-branch fusion depends on dataset scale and operator bias; it is therefore necessary to analyze accuracy, stability, and efficiency jointly rather than reporting only the single best score. Appendix A further gives a simplified linearized derivation showing why residual reuse can slow the collapse associated with over-smoothing.

5 Datasets

To comprehensively evaluate the effectiveness of ECR-GNN, we conduct experiments on four benchmark datasets from the TU Dortmund University (TUDataset) collection [42]. These datasets vary in size, domain, and graph characteristics, providing a robust testbed for evaluating cross-residual mechanisms and multi-operator architectures.

5.1 Dataset Overview

We use the following datasets in our experiments:

MUTAG MUTAG is a standard benchmark dataset for molecular property prediction, consisting of 188 chemical compounds representing nitroaromatic molecules. Each graph corresponds to a molecule, where nodes represent atoms and edges represent chemical bonds. The task is binary classification: predicting whether a molecule has mutagenic effects on the Gram-negative bacterium *Salmonella typhimurium*. This dataset is widely used in graph classification literature due to its small size and well-defined task, making it suitable for controlled ablation studies.

DD The DD dataset contains 1,178 protein structure graphs. Each graph represents a protein, where nodes correspond to amino acids and edges reflect spatial proximity relations. The task is binary classification: distinguishing enzymes from non-enzymes. Compared with the molecular datasets, DD contains much larger and structurally sparser graphs, making it a challenging benchmark for models that rely on compact graph-level summaries.

MSRC_9 MSRC_9 is a computer-vision graph classification dataset containing 221 segmented images from the Microsoft Research Cambridge collection. Each graph is a region adjacency graph in which nodes represent segmented image regions and edges encode spatial adjacency between neighboring regions. The task is multi-class classification over 8 categories. Compared with the molecular benchmarks, MSRC_9 emphasizes global spatial arrangement and inter-region context, making it useful for testing whether residual information reuse can preserve discriminative structure across multiple semantic regions.

Table 1: Summary of graph classification datasets used in experiments

Dataset	# Graphs	# Classes	Feature Dim.	Domain	Task Type
MUTAG	188	2	Dataset-specific TU features	Bio	Binary
DD	1,178	2	Dataset-specific TU features	Bio	Binary
MSRC_9	221	8	Dataset-specific TU features	Vision	Multi-class
AIDS	2,000	2	Dataset-specific TU features	Bio	Binary

AIDS AIDS is a bioinformatics dataset containing 2,000 chemical compounds screened for activity against HIV. Each graph represents a molecule, with nodes as atoms and edges as chemical bonds. The task is binary classification: predicting whether a compound is active against HIV. This is the largest dataset in our experiments, challenging the model's scalability and ability to learn from diverse molecular structures.

5.2 Dataset Statistics

Table 1 summarizes the key statistics of the datasets used in our experiments. Statistics are computed from the TUDataset loader in PyTorch Geometric.

5.3 Data Loading and Configuration

All datasets are obtained from the TUDataset collection through PyTorch Geometric [43].

Dataset Source We use the standard benchmark splits generated from the four named datasets MUTAG, DD, MSRC_9, and AIDS, all accessed through the same TUDataset interface.

Graph Representation Each graph is represented as a PyTorch Geometric `Data` object containing:

- **x**: Node feature matrix of shape $(n \times d_{\text{in}})$, where n is the number of nodes and d_{in} is the input feature dimension specific to the dataset
- **edge_index**: Graph connectivity in COO format, shape $(2, |\mathcal{E}|)$, where $|\mathcal{E}|$ is the number of edges
- **y**: Graph-level label (scalar for binary/multi-class classification)
- **batch**: Batch assignment vector (for mini-batch processing)

Node Features Node features are provided by the TUDataset loader in the form exposed by PyTorch Geometric. No additional feature engineering or normalization is applied during data loading. The feature dimension d_{in} is dataset-specific and automatically inferred by the loader. In practice, these inputs are derived from the node labels or node attributes available in each dataset, so the effective feature semantics differ across molecular, protein, and region-adjacency graphs.

Edge Attributes and Self-Loops **Edge attributes** are not used in our experiments. All graph convolution operators (GCN, GAT, Transformer) operate on unweighted graphs. **Self-loops** are added automatically by specific operators when required (e.g., GCN adds self-loops via $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$), rather than through a separate preprocessing step.

5.4 Data Splitting Protocol

We use 5-fold cross-validation for all datasets to ensure robust evaluation and enable fair comparison across model variants.

5-Fold Cross-Validation Each dataset is randomly shuffled once using a fixed seed and then partitioned into 5 equal-sized folds. For each fold $k \in \{0, 1, 2, 3, 4\}$:

1. Fold k is held out as the test set
2. The remaining 4 folds are concatenated to form the training set
3. The model is trained on the training set and evaluated on the test set
4. This process is repeated for all 5 folds, and the mean accuracy and standard deviation across folds are reported

The split is implemented as:

$$\begin{aligned} \text{train\_dataset} &= \mathcal{D}[0 : k \cdot \text{split\_len}] \cup \mathcal{D}[(k+1) \cdot \text{split\_len} : |\mathcal{D}|] \\ \text{test\_dataset} &= \mathcal{D}[k \cdot \text{split\_len} : (k+1) \cdot \text{split\_len}] \end{aligned} \tag{19}$$

where $\mathcal{D}$ is the full dataset, $\text{split\_len} = |\mathcal{D}|/5$, and k is the fold index.

Reproducibility To ensure reproducibility, random seeds are set before training:

- PyTorch random seed: 1024
- NumPy random seed: 1024
- Python random seed: 1024
- CUDA random seed: 1024 (if GPU is available)

The dataset shuffling is deterministic given the fixed random seed, ensuring that the 5-fold splits are consistent across different experimental runs.

No Official Splits Used We do not rely on predefined benchmark splits. Instead, we generate a unified 5-fold evaluation protocol for all datasets so that every architecture is assessed under the same partitioning scheme. All results report mean accuracy and standard deviation computed from the 5 folds.

5.5 Data Preprocessing and Augmentation

Minimal Preprocessing Our framework applies minimal preprocessing to the raw datasets:

- **Dataset shuffling:** The dataset is shuffled once using the fixed random seed before creating 5-fold splits
- **Batching:** Graphs are grouped into mini-batches of size 32
- **No feature normalization:** Node features are used as-is without scaling or centering
- **No graph augmentation:** No edge dropout, node dropping, or subgraph sampling is applied

Batching Strategy Training and test graphs are processed with separate mini-batch loaders:

- **Training loader:** batch size 32 with shuffling at each epoch
- **Test loader:** batch size 32 without shuffling for deterministic evaluation

The batch size of 32 is used across all datasets and models.

Regularization While not data augmentation per se, we apply **dropout** regularization during training:

- Dropout probability: 0.6 (default, configurable via `--drop` argument)
- Applied after each message passing layer
- Applied before the final classifier

Dropout acts as a form of implicit data augmentation by randomly dropping node features during training, which helps prevent overfitting despite the small dataset sizes.

5.6 Evaluation Metrics

Following standard practice in graph classification literature, we evaluate model performance using:

Accuracy Accuracy is the primary evaluation metric, computed as the proportion of correctly classified graphs:

$$\text{Accuracy} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{i=1}^{|\mathcal{D}_{\text{test}}|} \mathbb{1}[\hat{y}_i = y_i] \tag{20}$$

where $\mathcal{D}_{\text{test}}$ is the test set, $\hat{y}_i$ is the predicted label, and y_i is the ground-truth label.

Aggregation Across Folds For each dataset, we report:

- **Mean accuracy:** $\mu = \frac{1}{5}\sum_{k=0}^{4} \text{acc}_k$, where acc_k is the accuracy on fold k
- **Standard deviation:** $\sigma = \sqrt{\frac{1}{5}\sum_{k=0}^{4}(\text{acc}_k - \mu)^2}$

These statistics are computed across the 5 folds, providing a robust measure of model performance and stability. Results are reported in the format "mean ± standard deviation" (e.g., "0.85 ± 0.03").

6 Experiments

This section is organized in a result-first manner: we first report the main benchmark results, then analyze a refreshed MUTAG + GCNConv subset in detail, and finally discuss stability and efficiency.

6.1 Experimental Settings

6.1.1 Data Sources and Consolidation

The benchmark combines the full experiment grid with a refreshed evaluation of the MUTAG + GCNConv subset. For the final analysis, the refreshed subset replaces the earlier measurements of the same setting so that each configuration is counted only once. The resulting benchmark contains 720 evaluated configurations in total.

6.1.2 Configuration Space

We evaluate six architectures:

1. **PlainGNN**: plain stacked message passing
2. **NodeResGNN**: single-branch residual reuse
3. **NodeCrossGNN**: two-branch node-level cross residual
4. **GraphCondGNN**: sequential graph-level conditioning
5. **GraphResGNN**: repeated graph-level residual refinement
6. **GraphCrossGNN**: graph-level cross residual between sequential block pairs

Each architecture is instantiated with three operators (GCNConv, GATConv, TransformerConv), five depths ($h \in \{1, 2, 3, 4, 5\}$), and two hidden dimensions (32 and 64). The full grid therefore contains $6 \times 3 \times 5 \times 2 \times 4 = 720$ evaluated settings across the four datasets MUTAG, DD, MSRC_9, and AIDS.

Table 2: Best 5-fold graph classification accuracy (mean ± std) for each architecture on each dataset under the final benchmark setting.

Model	MUTAG	DD	MSRC_9	AIDS
PlainGNN	0.822 ± 0.0404	0.618 ± 0.0211	**0.982 ± 0.0170**	0.810 ± 0.0213
NodeResGNN	**0.832 ± 0.0315**	0.611 ± 0.0316	**0.982 ± 0.0265**	0.812 ± 0.0068
NodeCrossGNN	0.811 ± 0.0705	0.617 ± 0.0222	0.977 ± 0.0144	0.812 ± 0.0218
GraphCondGNN	0.827 ± 0.0653	0.609 ± 0.0277	0.977 ± 0.0144	0.812 ± 0.0173
GraphResGNN	0.816 ± 0.0465	**0.619 ± 0.0384**	0.973 ± 0.0223	0.812 ± 0.0083
GraphCrossGNN	0.778 ± 0.0315	0.614 ± 0.0275	**0.982 ± 0.0170**	**0.815 ± 0.0172**

Table 3: Best configuration of each architecture on the refreshed MUTAG + GCNConv subset.

Model	Dim	Depth	Acc.	Std.	Time (s)
PlainGNN	32	1	0.800	0.0930	577.4
NodeResGNN	32	4	0.811	0.0419	1018.8
NodeCrossGNN	64	2	0.811	0.0705	1842.4
GraphCondGNN	32	2	0.827	0.0653	1228.7
GraphResGNN	32	2	0.805	0.0733	1229.5
GraphCrossGNN	32	1	0.778	0.0315	1634.5

6.1.3 Training Protocol

All runs use the same training protocol: 5-fold cross-validation, Adam optimizer, learning rate 0.01, weight decay 10^{-2}, dropout 0.6, and up to 1500 epochs with early stopping once the training loss falls below 0.001. We report mean accuracy, fold standard deviation, and total execution time.

6.2 Main Results

Table 2 summarizes the best configuration of each architecture on each dataset. Figure 2 shows the average accuracy over all 720 evaluated configurations, while Figure 3 visualizes the best accuracy reached by each architecture on each dataset.

Three observations stand out.

Residual reuse is the most reliable gain. Averaged over the full configuration space, NodeResGNN achieves the best mean accuracy of 0.775, ahead of NodeCrossGNN (0.766), GraphCondGNN (0.748), PlainGNN (0.740), and GraphCrossGNN (0.734). This ranking indicates that preserving intermediate representations is more consistently beneficial than simply increasing branch count or graph-level routing complexity.

The gain pattern follows dataset structure rather than a single universal winner. On MUTAG, the best result is reached by NodeResGNN, which is consistent with the benefit of stabilizing shallow-to-medium-depth feature reuse on small molecular graphs. On AIDS, GraphCrossGNN attains the highest single score, but the average spread between architectures is narrow, indicating that this benchmark rewards graph-level interaction only in a limited regime. Most importantly, MSRC_9 shows the clearest architecture sensitivity in the mean-result view: NodeResGNN and NodeCrossGNN substantially outperform PlainGNN on average, which is consistent with the intuition that region-adjacency graphs depend on coordinated multi-region

Table 4: Efficiency comparison on MUTAG with GCNConv, dim=32, and depth=2. Relative overhead is measured against PlainGNN.

Model	Time (s)	Relative Overhead	Accuracy
PlainGNN	709.8	+0.0%	0.708
NodeResGNN	738.9	+4.1%	0.795
NodeCrossGNN	1158.0	+63.1%	0.724
GraphCondGNN	1228.7	+73.1%	0.827
GraphResGNN	1229.5	+73.2%	0.805
GraphCrossGNN	2187.2	+208.1%	0.622

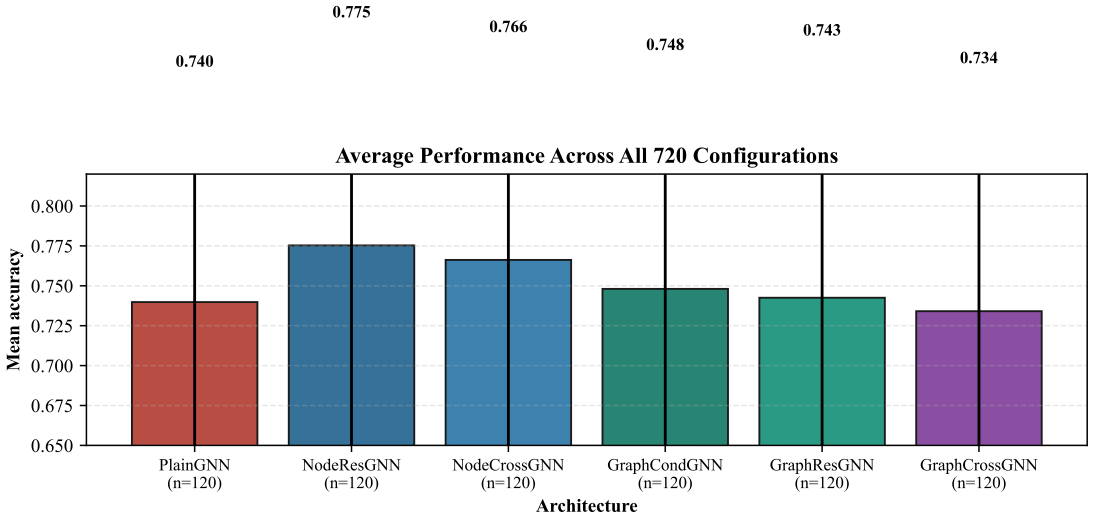

Figure 2: Average accuracy across all 720 evaluated configurations. NodeResGNN achieves the highest global mean accuracy (0.775), followed by NodeCrossGNN (0.766).

patterns and therefore benefit from stronger local-global information coupling. By contrast, DD shows only marginal separation between model families, suggesting that very large and sparse protein graphs are harder to improve through the current residual-readout design alone.

Dataset difficulty and structural match jointly determine the outcome. The mean accuracy over all settings is 0.898 on MSRC_9, 0.799 on AIDS, 0.719 on MUTAG, and only 0.587 on DD. However, high absolute accuracy alone does not explain the observed pattern. The larger average gains on MSRC_9 and MUTAG indicate that architectural choices matter when useful information must be preserved across multiple depths or multiple semantic regions, whereas the near-tied DD results suggest that the present framework is less suited to benchmarks dominated by large, sparse, and structurally heterogeneous graphs. This interpretation is an inference from the measured performance pattern together with the known benchmark characteristics.

6.3 Refreshed MUTAG + GCNConv Analysis

This refreshed subset is especially useful because it covers the full MUTAG + GCNConv setting with both widths and all depths. Table 3 lists the best configuration of each architecture on this subset, and Figure 4 shows the full depth curves for both widths.

This subset reveals a more nuanced pattern than a single headline score would suggest.

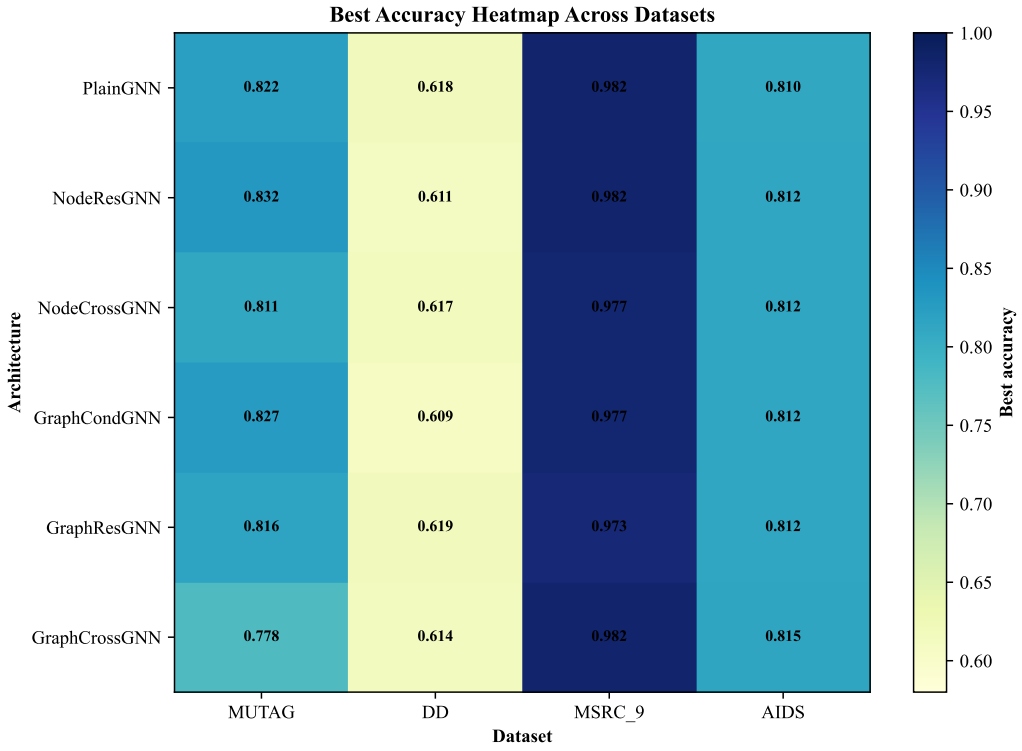

Figure 3: Best accuracy heatmap across datasets. MSRC_9 is consistently easy, DD remains the hardest benchmark, and AIDS shows only a narrow spread between the strongest architectures.

Graph-level conditioning can be strongest at shallow depth. On the dim=32 branch, GraphCondGNN peaks at 0.827 with depth 2, outperforming all other GCN-based variants in that setting. GraphResGNN is also strong at depth 2 (0.805), which means graph-level residual reuse is genuinely effective when the graph summary remains informative and the model is not yet deep enough to over-compress it.

NodeResGNN is the most depth-tolerant architecture in the refreshed subset. On the same dim=32 setting, PlainGNN drops from 0.800 at depth 1 to 0.600 at depth 5, a 25.0% relative degradation. In contrast, NodeResGNN reaches its peak at depth 4 (0.811) and still retains 0.773 at depth 5, showing that intra-branch residual accumulation is the most reliable safeguard against performance collapse.

Node-level cross residual is competitive but less stable. On the dim=64 setting, NodeCrossGNN reaches 0.811 at depth 2, matching the best GCN result of NodeResGNN. However, its fold standard deviation at that point is 0.070, clearly larger than the 0.030 observed for NodeResGNN in the same setting. Cross-branch exchange can therefore raise the ceiling, but it does not automatically improve robustness.

6.4 Stability and Efficiency

To separate average accuracy from robustness, Figure 5 plots the five-fold distribution on the shared MUTAG + GCNConv setting with dim=64 and depth=2. Table 4 reports execution time and relative overhead on the commonly referenced MUTAG + GCNConv setting with dim=32 and depth=2.

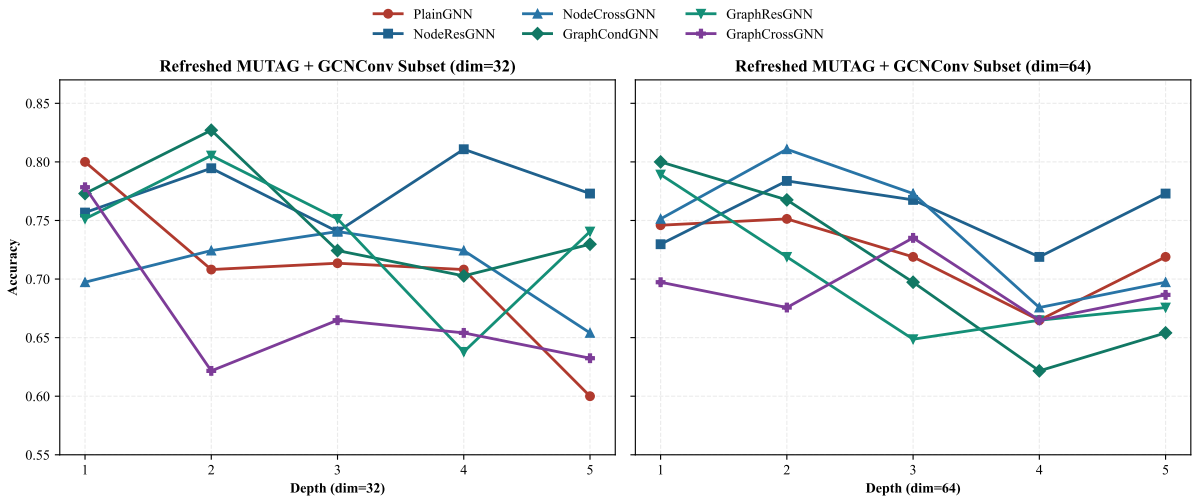

Figure 4: Depth sensitivity on the refreshed MUTAG + GCNConv subset. Dim=32 and dim=64 show different winners, which confirms that architecture comparisons should not rely on a single depth-width setting.

Two complementary conclusions follow.

NodeResGNN is the most stable architecture overall. Across all 720 evaluated configurations, it has the lowest mean fold standard deviation (0.0312), narrowly ahead of NodeCrossGNN (0.0316), and clearly better than PlainGNN (0.0516). The box plot shows the same behavior on the shared MUTAG + GCNConv setting: NodeResGNN keeps a tight fold range while remaining highly competitive in mean accuracy.

Efficiency sharply penalizes the more complex graph-level variants. On MUTAG + GCNConv with dim=32 and depth=2, NodeResGNN improves accuracy from 0.708 to 0.795 with only 4.1% extra time. GraphCondGNN reaches the strongest accuracy on that setting (0.827) but needs 73.1% more time, while GraphCrossGNN is both slower (+208.1%) and weaker (0.622). The full-grid timing statistics tell the same story: average execution time rises from 9136 seconds for PlainGNN to 9572 seconds for NodeResGNN, but jumps to 31266 seconds for GraphCrossGNN. In other words, the simplest residual mechanism offers the best default trade-off, whereas graph-level cross interaction should be reserved for tasks where the data structure clearly rewards it.

6.5 Summary of Findings

The full benchmark analysis supports four final claims. First, residual reuse is a stronger default design choice than branch proliferation. Second, cross-branch interaction is best understood as a targeted mechanism for graph classification settings that require sustained coupling between local patterns and global arrangement, with MSRC_9 providing the clearest evidence for this behavior in the current benchmark. Third, graph-level residual propagation is useful, but mainly in shallow or otherwise dataset-dependent regimes rather than as a universal winner. Fourth, once accuracy, stability, and execution time are considered together, NodeResGNN offers the best overall trade-off in the current study, while the more complex graph-level cross variants should be treated as specialized models whose benefits depend on structural fit to the data.

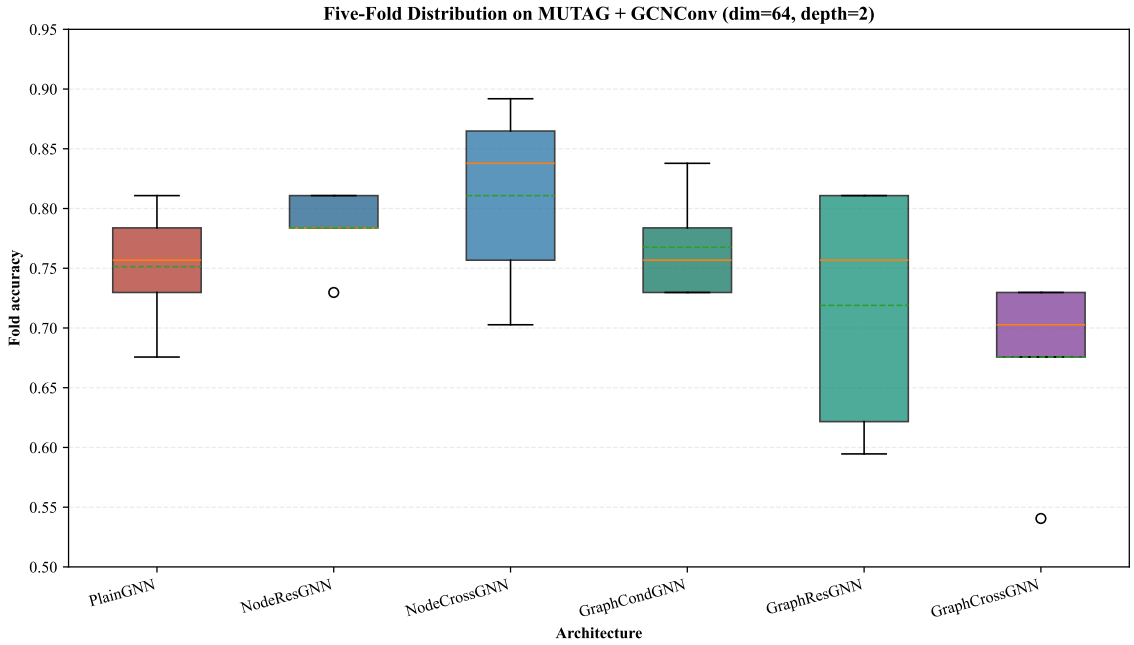

Figure 5: Fold-wise accuracy distribution on MUTAG + GCNConv (dim=64, depth=2). NodeResGNN has the tightest spread, whereas NodeCrossGNN attains a high mean with noticeably larger variance.

7 Conclusion

This paper studied ECR-GNN as a family of residual and cross-residual graph classification architectures and evaluated it under a unified benchmark setting covering 720 configurations.

The empirical picture is more conditional than a single headline result suggests. NodeResGNN is the strongest all-around design: it achieves the best global mean accuracy (0.775), the lowest average fold variance, and only a small execution-time increase over PlainGNN. NodeCrossGNN can reach similarly strong peak scores on selected settings, but it is less stable. More importantly, the benchmark comparison suggests that cross-residual interaction is most valuable when graph labels depend on coordinated multi-region or multi-substructure patterns rather than on a single dominant local cue. In the current experiments, MSRC_9 provides the clearest support for this interpretation, because residual and cross-residual variants obtain the largest average improvement over the plain baseline there. By contrast, the DD results remain tightly clustered, showing that the current design is less effective on very large and sparse protein graphs. Graph-level interaction is still useful in selected settings, including the best AIDS subset and shallow MUTAG configurations, but it is not universally superior and becomes expensive when implemented through multiple sequential block models.

The refreshed MUTAG + GCNConv subset adds an important nuance: the locally best architecture within one operator-dataset setting is not always the globally best architecture across the full benchmark. GraphCondGNN is strongest on the shallow dim=32 setting, whereas NodeResGNN remains the most depth-tolerant and stable model overall. This reinforces the main practical takeaway of the paper: cross-residual design should be judged jointly by structural fit, accuracy, robustness, and cost, not by a single best-case number.

Future work can move in three directions. First, the graph-level variants should be redesigned to reduce their substantial runtime overhead. Second, heterogeneous operator combinations deserve a direct study, since the current experiments still keep the operator type homogeneous within each model instance. Third, larger datasets and stronger statistical testing are needed to determine when cross-branch interaction offers a meaningful advantage beyond the simpler residual baseline.

A Why Residual Reuse Can Slow Over-smoothing

This appendix provides a simplified derivation that explains why residual reuse can mitigate over-smoothing. The goal is not to claim a full theorem for the nonlinear models in Section 4, but to show that the proposed residual pathways change the effective propagation filter from a single high-order power to a lower-order polynomial mixture.

A.1 Linearized Plain Propagation

Consider the standard linearized message-passing update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}\mathbf{H}^{(\ell)}, \tag{21}$$

where $\mathbf{S}$ is a normalized graph propagation operator and $\mathbf{H}^{(0)} = \mathbf{X}$. Ignoring nonlinearities and trainable weights yields

$$\mathbf{H}^{(\ell)} = \mathbf{S}^{\ell}\mathbf{X}. \tag{22}$$

If $\mathbf{S}$ is diagonalizable as $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1}$, then

$$\mathbf{H}^{(\ell)} = \mathbf{U}\mathbf{\Lambda}^{\ell}\mathbf{U}^{-1}\mathbf{X}. \tag{23}$$

For normalized propagation, the dominant eigenvalue is typically close to 1, while the remaining eigenvalues satisfy $|\lambda_i| < 1$. As ℓ grows, the factors λ_i^{ℓ} suppress non-dominant components, so node embeddings gradually collapse toward a low-dimensional smooth subspace. This is the standard over-smoothing effect: nodes become increasingly difficult to distinguish because repeated propagation attenuates the higher-frequency components that encode local differences.

A.2 Node-Level Residual Reuse

Now consider a residual reuse update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}\big(\mathbf{H}^{(\ell)} + \beta\mathbf{H}^{(\ell-1)}\big), \tag{24}$$

where $\beta \geq 0$ controls the strength of the reused hidden state. The implementation studied in this paper corresponds to $\beta = 1$.

Define $\mathbf{H}^{(\ell)} = P_{\ell}(\mathbf{S})\mathbf{X}$ with $P_0(\mathbf{S}) = \mathbf{I}$ and $P_1(\mathbf{S}) = \mathbf{S}$. Then Eq. (24) implies the polynomial recurrence

$$P_{\ell+1}(\mathbf{S}) = \mathbf{S}\big(P_{\ell}(\mathbf{S}) + \beta P_{\ell-1}(\mathbf{S})\big). \tag{25}$$

The first few terms are

$$P_1(\mathbf{S}) = \mathbf{S}, \tag{26}$$

$$P_2(\mathbf{S}) = \mathbf{S}^2 + \beta\mathbf{S}, \tag{27}$$

$$P_3(\mathbf{S}) = \mathbf{S}^3 + 2\beta\mathbf{S}^2, \tag{28}$$

$$P_4(\mathbf{S}) = \mathbf{S}^4 + 3\beta\mathbf{S}^3 + \beta^2\mathbf{S}^2. \tag{29}$$

Therefore, residual reuse no longer applies only the single power $\mathbf{S}^\ell$; instead, it produces a mixture of lower-order and higher-order propagation terms. In spectral coordinates, each eigencomponent evolves as

$$z_i^{(\ell+1)} = \lambda_i\big(z_i^{(\ell)} + \beta z_i^{(\ell-1)}\big), \tag{30}$$

which is fundamentally different from the plain recurrence $z_i^{(\ell)} = \lambda_i^\ell z_i^{(0)}$. The plain model attenuates every non-dominant mode with a pure monomial factor, whereas the residual model preserves a polynomial combination of powers of λ_i. As a result, informative mid-frequency components are not suppressed as aggressively at the same depth.

A.3 Implication for Over-smoothing

The derivation above suggests the following interpretation. Plain stacking behaves like a repeated low-pass filter whose order increases monotonically with depth. Residual reuse changes that filter into a polynomial response that still smooths the graph, but preserves more low-order propagation information from earlier layers. This does not eliminate smoothing, yet it slows the rate at which node representations collapse to an almost indistinguishable subspace. In other words, residual reuse can improve depth tolerance because it reduces the dominance of the highest propagation order.

A.4 Graph-Level Residual Passing

The same intuition extends to graph-level residual propagation. In the graph-conditioned variants, a pooled summary from the previous block is broadcast back to the current block, so the hidden state is no longer determined only by repeated applications of $\mathbf{S}$. A simplified affine form is

$$\mathbf{H}_t^{(\ell+1)} = \mathbf{S}\mathbf{H}_t^{(\ell)} + \mathbf{1}\big(\mathbf{g}_{t-1}\big)^\top, \tag{31}$$

where $\mathbf{g}_{t-1}$ is the preceding graph summary. The additive term injects a lower-order global context at every block and prevents the representation from being governed solely by deeper powers of the propagation operator. This offers a second pathway for delaying over-smoothing, although the empirical sections show that graph-level reuse is more sensitive to dataset structure and computational cost than the lighter node-level residual design.

A.5 Scope of the Derivation

This appendix uses a linearized analysis and therefore abstracts away nonlinear activations, dropout, optimizer dynamics, and operator-specific details. It should be read as an explanatory

mechanism rather than as a complete proof. Still, it aligns with the empirical observation that node-level residual reuse remains more stable than plain stacking as depth increases, and it provides a compact theoretical rationale for why reusing earlier states can alleviate over-smoothing.

References

[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[3] V. P. Dwivedi and X. Bresson, "A generalization of transformer networks to graphs," *arXiv preprint arXiv:2012.09699*, 2020.

[4] G. Li, E. Muller, A. Thabet, and B. Ghanem, "Deeper insights into graph convolutional networks for semi-supervised learning," *arXiv preprint arXiv:1809.02584*, 2018.

[5] Y. Chen, X. Wei, P. Xu, X. Wang, P. Luo, Z. Li, and J. Tang, "Revisiting over-smoothing in deep gcns," *arXiv preprint arXiv:2003.13663*, 2020.

[6] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations (ICLR)*, 2019.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[8] X. Bresson and T. Laurent, "Residual gated graph convnets," *arXiv preprint arXiv:1711.07553*, 2017.

[9] L. Zhao and L. Akoglu, "Pairnorm: Tackling over-smoothing in gns," *arXiv preprint arXiv:2009.10698*, 2020.

[10] T. Cai, J. Luo, S. Wang, K. Zhang, Y. Tian, L. Yang, Z. Li, and K. Weinberger, "Graphnorm: A principled approach to accelerating graph neural network training," in *International Conference on Machine Learning*, 2021, pp. 1277–1287.

[11] Y. Rong, W. Huang, T. Xu, and J. Huang, "Dropedge: Towards deep graph convolutional networks on node classification," *arXiv preprint arXiv:1907.10903*, 2019.

[12] K. Xu, C. Li, Y. Tian, X. Du, J. Leskovec, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *International Conference on Machine Learning*, 2018, pp. 5449–5458.

[13] H. Du, Y. Zhang, P. Zhang, B. Gu, X. Wei, J. Guo, A. Li, Z. Wang, T. Ma *et al.*, "Densegnn: universal and scalable deeper graph neural networks for high-performance property prediction in crystals and molecules," *npj Computational Materials*, vol. 10, no. 1, p. 304, 2024.

[14] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Advances in neural information processing systems*, 2018, pp. 4800–4810.

[15] J. Lee, I. Lee, and J. Kang, "Self-attention graph pooling," *arXiv preprint arXiv:1904.08082*, 2019.

[16] M. Gori, G. Monfardini, and F. Scarselli, "A new model for learning in graph domains," *Proceedings of IJCNN*, 2005.

[17] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *International Conference on Machine Learning*, 2017, pp. 1263–1272.

[18] D. K. Duvenaud, D. Maclaurin, J. Iparraguirre, R. Bombarell, T. Hirzel, A. Aspuru-Guzik, and R. P. Adams, "Convolutional networks on graphs for learning molecular fingerprints," in *Advances in neural information processing systems*, 2015, pp. 2224–2232.

[19] C. Cangea, P. Veličković, N. Jovanović, T. Kipf, and P. Liò, "Towards sparse hierarchical graph classifiers," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2018, pp. 165–176.

[20] C. Liu, X. Wang, Y. Li, Y. Wang, and J. Gao, "Graph pooling for graph neural networks," in *International Joint Conference on Artificial Intelligence*, 2023, pp. 4254–4261.

[21] Z. Li, H. Chen, Z. Zeng, J. He, X. Mao, N. Lv, and L. Chen, "Graph pooling for graph-level representation learning: a survey," *Artificial Intelligence Review*, vol. 57, no. 8, pp. 1–51, 2024.

[22] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in neural information processing systems*, 2017, pp. 1024–1034.

[23] M. Zhang, Y. Liu, Q. Chen, and S. Pan, "Personalized graph neural networks with attention mechanism," *arXiv preprint arXiv:1910.08887*, 2019.

[24] D. Beaini, S. Passaro, E. Létouzey, J. Domke, S. Erdos, P. Lió, P. Veličković, and M. M. Bronstein, "Directional graph networks," *arXiv preprint arXiv:2010.02863*, 2020.

[25] E. Rossi, F. Frasca, F. M. Bianchi, N. Alletto, and P. Lio, "Edge directionality improves learning on heterophilic graphs," in *International Conference on Learning Representations (ICLR)*, 2023.

[26] F. M. Bianchi, F. Grattarola, and C. Alippi, "Graph neural networks with convolutional arma filters," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

[27] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, G. Steeg, and A. Galstyan, "Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing," in *International Conference on Machine Learning*, 2019, pp. 33–42.

[28] A. Gravina, N. Alletto, A. Rossi, F. M. Bianchi, and P. Lio, "Anti-symmetric dgn: A stable architecture for deep graph networks," *arXiv preprint arXiv:2210.09789*, 2022.

[29] S. Li, M. Zhang, M. Jin, H. Zhang, R. Ying, Z. Li, and J. Yan, "A non-asymptotic analysis of oversmoothing in graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2023.

[30] J. Topping, F. Di Giovanni, B. Chamberlain, M. Bronstein, D. Vendrig, and P. Lio, "Understanding over-squashing and bottlenecks on graphs via curvature," *arXiv preprint arXiv:2111.14522*, 2021.

[31] U. Alon and E. Yahav, "On the bottleneck of graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2021.

[32] Y. Li, X. Wang, Z. Wang, and J. Li, "Learning graph normalization for graph neural networks," *Neurocomputing*, vol. 486, pp. 140–151, 2022.

[33] G. Li, E. Müller, A. Thabet, and B. Ghanem, "Training graph neural networks with 1000 layers," in *International Conference on Machine Learning*, 2021, pp. 6403–6413.

[34] K. Oono and T. Suzuki, "Simple and deep graph convolutional networks," *arXiv preprint arXiv:2006.13604*, 2020.

[35] G. Wang, I. Balazevic, M. Znis, M. Gitta, Y. Kwon, M. Seeger, X. Li, L. De Raedt, and M. Welling, "Multi-hop attention graph neural networks," in *International Joint Conference on Artificial Intelligence*, 2021, pp. 2112–2120.

[36] X. Sun, R. Wang, L. Cui, X. Kong, and S. Li, "Attention-based graph neural networks: a survey," *Artificial Intelligence Review*, vol. 56, no. 11, pp. 13 493–13 539, 2023.

[37] J. Cai, X. Zheng, C. Luo, C. Yang, and E. Santus, "Understanding graph embedding methods and their applications," *arXiv preprint arXiv:2012.08019*, 2020.

[38] H. Van Pham, U. The, T. Do, and A. Nguyen, "Hierarchical pooling in graph neural networks to enhance graph classification," *Applied Sciences*, vol. 11, no. 17, p. 8164, 2021.

[39] H. Gao and S. Ji, "Graph u-net," *arXiv preprint arXiv:1905.05178*, 2019.

[40] J. Klicpera, A. Bojchevski, and S. Günnemann, "Combining label propagation and simple models out-performs graph neural networks," *arXiv preprint arXiv:1810.05997*, 2018.

[41] L. Zhao, Z. Leng, Y. Chen, and L. Akoglu, "Tackling over-smoothing for general graph learning," *arXiv preprint arXiv:2008.09864*, 2020.

[42] N. Keriven, G. Peyré, and S. Vaiter, "A unified benchmark for the diagnosis and treatment of graph neural networks," *arXiv preprint arXiv:2012.01450*, 2020.

[43] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," *arXiv preprint arXiv:1903.02428*, 2019.